Prompt Library — Complete Step-by-Step Guide

A **Prompt Library** is an organized collection of reusable prompts. Instead of writing the same prompt repeatedly, you create it once, save it with useful information and reuse or improve it later.

For teaching Prompt Engineering, you can explain the concept as:

Create → Test → Evaluate → Store → Organize → Reuse → Improve

1. What is a Prompt Library?

A prompt library is similar to a **digital notebook/database for prompts**.

For example, you may create:

Prompt Library

```
│
├── Education
│   ├── Lecture Explanation
│   ├── Question Generation
│   ├── MCQ Generation
│   └── Assignment Evaluation
│
├── Programming
│   ├── Python Code
│   ├── C Programs
│   ├── Debugging
│   └── Code Explanation
│
├── Generative AI
│   ├── Prompt Engineering
│   ├── LLM
│   ├── RAG
│   └── Fine-Tuning
│
├── Research
│   ├── Literature Review
│   ├── Research Summary
│   └── Paper Analysis
│
```

└── Administration

├── Notice

├── Circular

└── Official Letter

This makes prompts easy to find and reuse.

2. Why Do We Need a Prompt Library?

Without a library:

Write prompt

↓

Use prompt

↓

Forget prompt

↓

Write again

With a library:

Create prompt

↓

Test prompt

↓

Evaluate prompt

↓

Save prompt

↓

Reuse whenever required

Benefits

- Saves time
- Avoids rewriting prompts
- Standardizes output
- Makes prompts reusable
- Allows version control
- Helps compare prompts
- Helps students learn prompt engineering
- Makes organizational AI usage more consistent

3. Where Can I Get Prompt Libraries?

There are two possibilities.

A. Existing online prompt libraries

You can find publicly available prompt collections from sources such as:

OpenAI Prompt Examples / Guides

OpenAI Prompt Engineering Guide

Useful for learning prompt-design techniques and examples.

Anthropic Prompt Library

Anthropic Prompt Library

It contains examples that demonstrate practical prompt patterns.

Google Gemini Prompting Guide

Google Gemini Prompting Guide

Useful for learning prompting strategies.

GitHub

You can also find many open-source prompt collections on GitHub by searching for:

prompt library

LLM prompts

prompt engineering

awesome prompts

AI prompts

4. Where Should YOU Store Your Prompt Library?

You don't necessarily need specialized software.

You can start with:

Level 1 — Excel

Very easy for students and faculty.

Prompt_ID | Category | Prompt_Name | Prompt | Model | Score | Version

Level 2 — Word/Google Docs

Good for documentation.

Level 3 — JSON

Good for programmers and applications.

Level 4 — Database

For a large organization:

MySQL

PostgreSQL

MongoDB

Level 5 — GitHub

Good for version control and collaborative development.

Level 6 — Custom Prompt Library Web Application

For example:

Browser

↓

Prompt Library Website

↓

Backend API

↓

Database

For your teaching purpose, I recommend starting with **Excel + JSON**, then moving to a database/web application.

5. How to Create Your Own Prompt Library

Let's create one from the beginning.

Step 1 — Identify categories

Create categories such as:

Education

Programming

Research

Generative AI

Data Science

Administration

Content Creation

Evaluation

6. Step 2 — Create a Prompt Template

Don't store only:

"Explain Machine Learning."

Instead create a reusable template:

Act as a {ROLE}.

Explain {TOPIC} to {TARGET_AUDIENCE}.

Use:

- Simple language
- Important concepts
- Real-world examples
- A summary at the end

Difficulty level: {LEVEL}

Maximum length: {LENGTH}

Now it can be reused.

For example:

ROLE = B.Tech Professor

TOPIC = Machine Learning

TARGET_AUDIENCE = First-year students

LEVEL = Beginner

LENGTH = 500 words

7. Step 3 — Give the Prompt an ID

Every prompt should have a unique ID.

Example:

EDU-001

Meaning:

EDU = Education

001 = Prompt number

Other examples:

PROG-001

GENAI-001

RESEARCH-001

ADMIN-001

8. Step 4 — Give the Prompt a Name

Instead of searching for the entire prompt, give it a meaningful name.

Example:

Beginner Topic Explanation

9. Step 5 — Store Metadata

A professional prompt library should store more than just the prompt.

I recommend these fields:

Field	Example
Prompt ID	EDU-001
Prompt Name	Beginner Topic Explanation
Category	Education
Description	Explains technical concepts
Prompt Template	Actual prompt
Variables	ROLE, TOPIC, AUDIENCE
Model	GPT / Claude / Gemini
Temperature	0.3
Version	1.0
Author	Faculty
Created Date	31-08-2026
Evaluation Score	92%
Status	Approved
Tags	education, explanation
Notes	Works well for beginners

10. Example Prompt Library in Excel

Create an Excel file called:

Prompt_Library.xlsx

Create columns:

Prompt ID	Category	Prompt Name	Prompt Template	Variables	Model	Score	Version
EDU-001	Education	Topic Explanation	Explain {TOPIC} to {AUDIENCE}...	TOPIC,AUDIENCE	GPT	92%	1.0
EDU-002	Education	MCQ Generator	Generate {N} MCQs on {TOPIC}...	N,TOPIC	GPT	89%	1.0
PROG-001	Programming	Code Generator	Write {LANGUAGE}	LANGUAGE,TASK	GPT	94%	1.2

Prompt ID	Category	Prompt Name	Prompt Template	Variables	Model	Score	Version
			code for...				

This is already a **prompt library**.

11. How to Store a Prompt in JSON

For programming applications, JSON is much better.

Example:

```
{
  "prompt_id": "EDU-001",
  "name": "Beginner Topic Explanation",
  "category": "Education",
  "description": "Explain technical topics to beginners",
  "template": "Act as a {ROLE}. Explain {TOPIC} to {AUDIENCE} using simple language and real-world examples.",
  "variables": [
    "ROLE",
    "TOPIC",
    "AUDIENCE"
  ],
  "model": "LLM",
  "version": "1.0",
  "score": 92,
  "status": "approved"
}
```

12. How to Use a Stored Prompt

Suppose the library contains:

EDU-001

Template:

Act as a {ROLE}.

Explain {TOPIC} to {AUDIENCE}

using simple language and real-world examples.

You provide:

ROLE = AI Professor

TOPIC = Transformers

AUDIENCE = B.Tech students

The program replaces the variables.

Final prompt:

Act as an AI Professor.

Explain Transformers to B.Tech students

using simple language and real-world examples.

Then send the resulting prompt to the LLM.

13. Simple Python Prompt Library

You can create a very simple library without a database.

```
prompt_library = {
    "EDU-001": {
        "name": "Beginner Topic Explanation",
        "category": "Education",
        "template": """
        Act as a {role}.
        Explain {topic} to {audience}
        using simple language and real-world examples.
        """
    },

    "PROG-001": {
        "name": "Programming Explanation",
        "category": "Programming",
        "template": """
        Act as an experienced programming instructor.
        Explain {topic} using {language}.
        Provide explanation, algorithm, code and example.
        """
    }
}
```

14. Retrieve a Prompt

```
prompt = prompt_library["EDU-001"]
```

```
print(prompt["name"])
print(prompt["template"])
```

Output:

Beginner Topic Explanation

15. Fill the Variables

```
template = prompt_library["EDU-001"]["template"]
final_prompt = template.format(
    role="AI Professor",
    topic="Transformers",
    audience="B.Tech students"
)
print(final_prompt)
```

The result becomes:

Act as an AI Professor.
Explain Transformers to B.Tech students
using simple language and real-world examples.

16. Send It to an LLM

Conceptually:

Prompt Library
↓
Retrieve Prompt
↓
Fill Variables
↓
Final Prompt
↓
LLM API
↓
Response

For example:

```
response = client.responses.create(
    model="your-model",
    input=final_prompt
```

)

print(response.output_text)

The exact API call depends on the model/provider you use.

17. Add Evaluation to Your Library

This is a very important step.

Don't just store:

Prompt

Store:

Prompt + Test Dataset + Evaluation Metrics + Score

For example:

Prompt	BERTScore	ROUGE	Human Score	Overall
EDU-001 v1	0.86	0.72	85%	84%
EDU-001 v2	0.91	0.78	93%	91%

Now you know that:

Version 2 performs better.

18. Prompt Versioning

Never overwrite a good prompt immediately.

Use:

EDU-001 v1.0

EDU-001 v1.1

EDU-001 v2.0

For example:

Version 1.0

Explain Machine Learning.

Version 1.1

Explain Machine Learning in simple English.

Version 2.0

Act as a B.Tech professor.

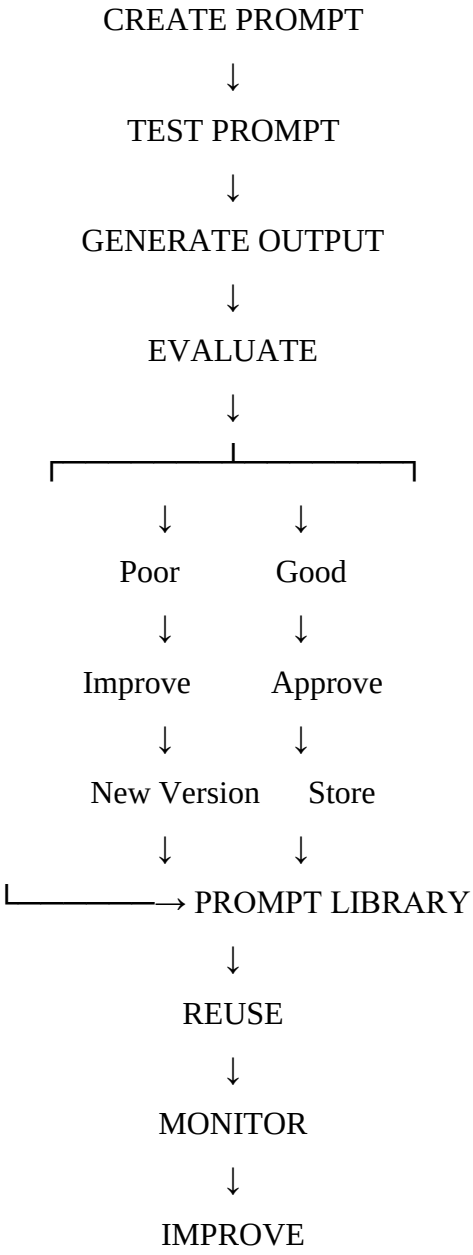
Explain Machine Learning to first-year students.

Use simple English, examples and a summary.

Evaluate each version.

19. Prompt Library Workflow

A professional workflow is:



20. Recommended Folder Structure

If you use GitHub or your computer:

Prompt-Library/

```
│
├── README.md
│
├── prompts/
│   │
│   ├── education/
│   │   ├── topic_explanation.json
```

```
│   │   ├── mcq_generator.json
│   │   └── question_generator.json
│   │
│   ├── programming/
│   │   ├── python_code.json
│   │   └── debugging.json
│   │
│   ├── generative_ai/
│   │   ├── prompt_engineering.json
│   │   └── llm_explanation.json
│   │
│   └── research/
│       ├── literature_review.json
│       └── paper_summary.json
│
├── datasets/
│   └── evaluation_questions.csv
│
├── evaluations/
│   └── prompt_scores.csv
│
└── versions/
```

This is a good structure for a **college Prompt Engineering project**.

21. How to Find a Prompt in Your Library

Suppose you have 500 prompts.

Don't manually search through them.

Use:

Search:

"Python debugging"

The library returns:

PROG-015

Python Debugging Assistant

Or search by:

Category = Programming

Tag = Python

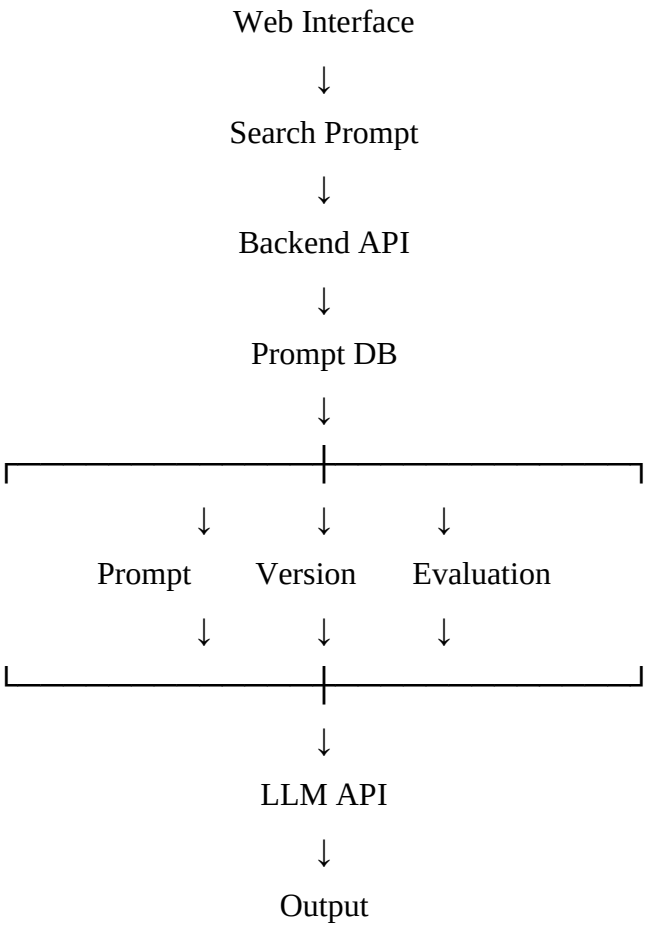
Difficulty = Advanced

Score > 90%

Then you get the best prompts.

22. A More Advanced Prompt Library

Eventually you can build:



Database tables could be:

PROMPTS

---

prompt_id

name

category

template

author

created_date

VERSIONS

---

version_id

prompt_id

version

template

date

EVALUATIONS

---

evaluation_id

prompt_id

version

metric

score

23. What Should You Start With?

For your **Prompt Engineering teaching/lab**, I recommend this progression:

Level 1

Excel Prompt Library

Learn:

- Prompt ID
- Category
- Prompt
- Variables
- Version
- Score

Level 2

JSON Prompt Library

Learn:

- JSON
- Python
- Prompt templates

- Variable substitution

Level 3

GitHub Prompt Library

Learn:

- Git
- Version control
- Collaboration
- Prompt versioning

Level 4

Database Prompt Library

Learn:

- MySQL/PostgreSQL
- CRUD operations
- Search
- Filtering

Level 5

Web-Based Prompt Library

Learn:

HTML/CSS/JavaScript + Python Flask/FastAPI + MySQL + LLM API

24. Best Structure for Your Own Library

If you are starting now, I suggest creating:

AI_Prompt_Library.xlsx

with these sheets:

Sheet 1 — Prompt Library

ID Category Name Template Variables Version Status

Sheet 2 — Test Cases

Test ID Prompt ID Input Expected Output

Sheet 3 — Evaluation

Prompt ID Accuracy BERTScore ROUGE BLEU Human Score Overall

Sheet 4 — Prompt Versions

Prompt ID Version Changes Date Score

Sheet 5 — Categories

Category ID Category Description

This turns a simple collection of prompts into a **proper Prompt Management and Evaluation System**.

One-line definition for students

A Prompt Library is a structured collection of reusable, tested, evaluated, and version-controlled prompts that can be stored, searched, modified, and reused for different AI tasks.

And the complete process is:

Create → Store → Retrieve → Customize → Execute → Evaluate → Version → Reuse.